CONSUMER INSIGHT PORTAL

# How the Forsta ETL works

From a survey in Forsta to verified, report-ready numbers on the portal and in the team's inbox. Each stage is explained step by step, with every technical word defined where it first appears.

① Source: Forsta · ② Schedule · ③ API · ④ Data received · ⑤ Extract · ⑥ Transform · ⑦ Load · ⑧ Verify · ⑨ Derive & publish · ⑩ Failures · ⑪ Timing & memory

Version 1.0 · 13 October 2026 · **Updated 23 Oct 2026** (version 1.1: Monday watch schedule, stricter load, value-by-value checks, serving rows, dashboards, publishing, team email, measured timing and memory) · Facts taken from the code (`etl/`, `scripts/`) and the live database

# How to read this document

Each **keyword** ⓘ *a technical term you may not know* is **bold with a yellow underline**, and its meaning follows straight after it in a yellow note, like this one. You never need to turn to a glossary.

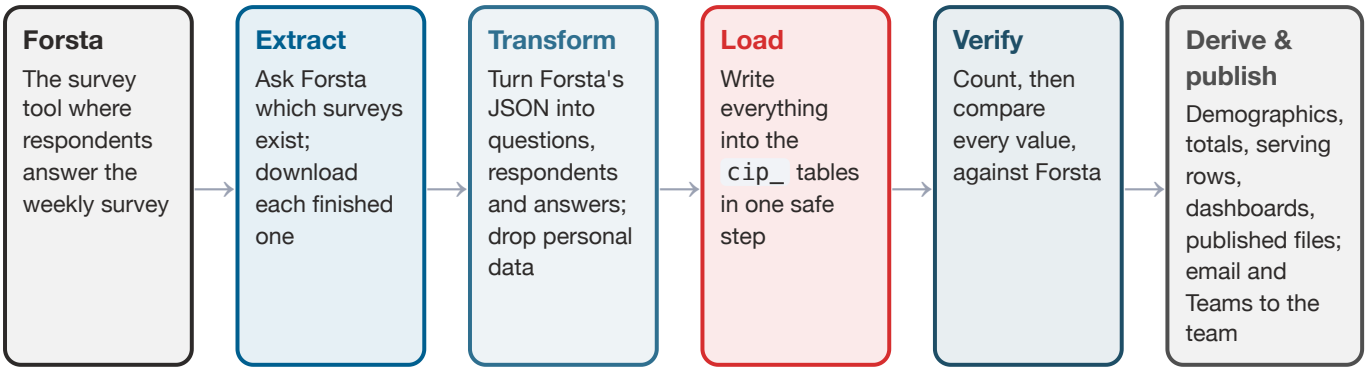
Source Extract Transform Load Verify Derive Failure

Each stage keeps the same colour in every heading, diagram and table.


1. The whole pipeline on one page
2. Where the data comes from: Forsta
3. When it runs: frequency and timing
4. The Forsta API: what we call
5. What data we receive
6. Extract: finding and fetching surveys
7. Transform: turning JSON into rows
8. Load: writing to the database
9. Verify: proving nothing was lost
10. Derive: making it report-ready, publishing, telling the team
11. What we ingest and what we never store
12. When something goes wrong
13. Worked example: one wave, start to finish
14. Timing and memory (measured 23 Oct 2026)


# 1. The whole pipeline on one page

**ETL** ⓘ *Extract, Transform, Load: take data out of one system, reshape it, and put it into another* is the program that copies our consumer survey answers from Forsta into our database, and on to the portal and the team's inbox. It runs on its own: on each planned launch Monday it watches Forsta every 30 minutes until that week's wave is in, and every Friday it does one catch-all run.



| Stage | What actually happens | Code |
|---|---|---|
| Source | Respondents from a panel answer a survey in Forsta. When the target of about 400 qualified people is reached, the survey is **closed**. | — |
| Extract | Gets the list of every survey. Each survey is marked *testing*, *due*, *loaded*, *failed*, *hibernated* or *too small*. For each *due* survey it downloads the questionnaire and every response. | `etl/forsta_etl.py` `etl/forsta_client.py` |
| Transform | Works out each question's type (single choice, tick-all, grid, number, text). Converts every answer from text to the right type. Removes or scrambles personal fields. | `etl/forsta_api.py` `etl/records.py` |
| Load | Writes the survey, its questions, its respondents and their answers. Respondents and answers are written together, so they either all succeed or all fail. Any warning from the database fails the load instead of silently changing a value. | `etl/loaders.py` `app/core/database.py` |
| Verify | Checks that the numbers of respondents, questions and answers match what Forsta sent. Only then does the wave appear in reports. The weekly run then compares every stored answer with Forsta, value by value, and runs the integrity checks. | `verify()`, `reconcile_survey()` in `etl/forsta_etl.py`; `etl/integrity.py` |
| Derive | Builds age and income bands, regions, cross-wave question links, pre-counted totals, search, the data team's line-by-line table, the serving rows every screen reads (with significance flags), the topic dashboards and the published files. Then it emails and posts to the team. | `etl/run_pipeline.py` `app/data/` `etl/serving.py` `api/dashboards.py` `etl/publish.py` `etl/notify.py` |

**In one sentence**

Every planned Monday evening the job starts asking Forsta "is this week's wave finished?" every 30 minutes. When it is, it downloads it in full, reshapes and stores it, proves nothing was lost, builds every finished number, publishes the portal's files and emails the team. A whole wave takes about 4–5.5 minutes end to end (section 14).

# 2. Where the data comes from: Forsta

**Forsta** ⓘ *the survey platform, formerly called Decipher, where Coresight's surveys are written, run and stored; our account is on the server se1.decipherinc.com* holds every survey Coresight runs. Respondents never touch our systems; they answer in Forsta.

## 2.1 The things Forsta holds

| Thing | What it is | Example |
|---|---|---|
| **Survey** | One questionnaire run once. For the weekly tracker, **one survey = one wave**. **Wave** ⓘ *one week's run of the same survey; waves are compared week to week* | "Shopping and Spending – inc Home Furnishings + Holiday Tracker + GenAI" |
| **Path** | **Path** ⓘ *Forsta's unique address for a survey, in the form selfserve/58f/YYMMNN, where 58f is the Coresight folder*. It is the key we use to track each survey. | `selfserve/58f/260908` |

| Thing | What it is | Example |
|---|---|---|
| **Respondent / record** | One person's attempt at the survey. **Record** ⓘ *Forsta's word for one respondent's row of answers* | record 412, uuid `a1b2…` |
| **Status** | How far the person got: **1** terminated (screened out), **2** over quota (the group was already full), **3** qualified (finished and counted), **4** partial (left before the end). | Weekly wave: ≈400 qualified out of 403–887 started |
| **Datamap** | **Datamap** ⓘ *Forsta's machine-readable description of the questionnaire: every question code, its wording, its type and every answer option with its number* | `D1` "What is your gender?" 1=Male, 2=Female… |
| **State** | The survey's lifecycle: *testing* → *live* (in field) → *closed*. | `"state": "closed"` |
| **Hibernated** | **Hibernated** ⓘ *Forsta moves old closed surveys (about 3–4 months after close) to cold storage; their data cannot be read until a person reactivates them in Forsta*. The API then answers **428**. | The 2025 holiday waves |

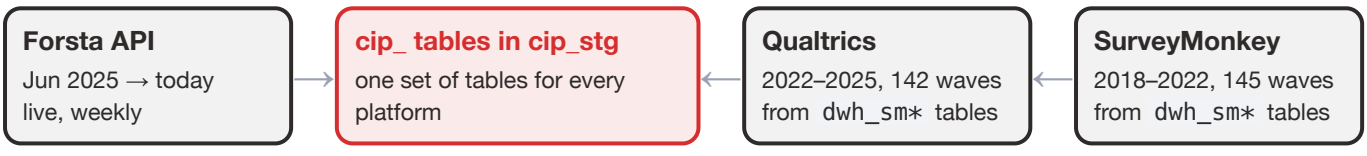
## 2.2 The surveys

| Kind | Pattern |
|---|---|
| **Weekly tracker** | "Shopping and Spending – inc …". Launched every **Monday**. It has 15 core questions (q1–q5 activities and purchases, CS1–CS2 outlook, D1–D8 demographics) plus that week's modules. |
| **Module** | **Module** ⓘ *an extra block of questions added to a wave, such as a retail sector or a topic like tariffs or GLP-1*. The lead sector rotates every 12 weeks: Drugstore → eCommerce → Luxury → Dept stores → Beauty → Home furnishings → Home improvement → Social commerce → Off-price → Apparel → Mass/warehouse → Footwear. |
| **Holiday tracker** | Questions coded `XM`, added to every weekly wave from late September to December. Waves are numbered within each season. |
| **Annual / ad hoc** | Annual studies (holiday shopping, back to school, online grocery…) and one-off client studies. Same pipeline, same tables. They field for days (Holiday 2025: 8 days), so they load **only once Forsta closes them**. Annual studies whose titles change from year to year are tied into one **family** ⓘ *the group of waves compared over time; see 6.4* by a list in `config/survey_map.yml` (Holiday, Online Grocery, Apparel, Back to School, Shopping and Social Media), so they compare year on year. |

## 2.3 Our access

We read Forsta with an **API key** ⓘ *a 64-character secret password for programs, stored in the git-ignored .env file and never printed or logged*. It belongs to user 879 in the "Coresight Research" folder (`58f`). The key can **only read**: our code has no way to edit, reactivate or delete anything in Forsta.

## 2.4 Other sources (history only)




**cip_stg** ⓘ *our own Azure MySQL staging database; every pipeline table in it starts with cip_. The old Qualtrics and SurveyMonkey source tables (dwh_sm*) stay in the shared dwh_stg database and are only read*. History was loaded once. Only Forsta is fetched on a schedule.

# 3. When it runs: frequency and timing

The job is started by **launchd** ⓘ *the macOS built-in scheduler, which runs a program at set times like cron on a server; if the Mac was asleep, it runs the missed slot when it wakes* through the agent `com.coresight.cip-forsta-etl`, which runs `scripts/weekly_forsta.py`.

## 3.1 The schedule (India time, IST)

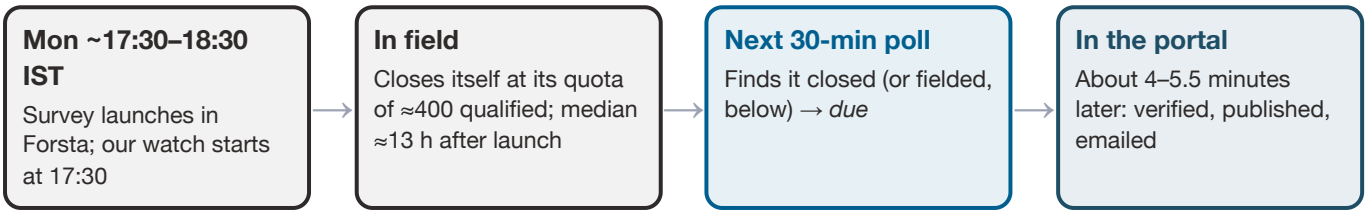
Since 23 Oct the job no longer runs at fixed times every day. It follows the data team's **launch schedule** ⓘ *the "Tracking by Week" tab of the team's workbook, imported into config/launch_schedule.yml: which Monday each lead sector and topic goes out*.

| When | What happens |
|---|---|
| **Planned Monday, 17:30** | A heads-up goes to Teams ("Forsta launches … today ~18:00 IST"). Then the **watch** ⓘ *one long-running job that repeats the normal run every 30 minutes* starts: a run every **30 minutes** until that week's wave is loaded **and** Forsta has closed it, for at most **54 hours** (Mon 17:30 → Wed 23:30). |
| **Friday, 09:00** | One catch-all run. It picks up annual and ad hoc studies, reopened surveys, and any wave the watch missed. Friday's run also always does the full integrity and drift checks. |
| Hourly | A separate **watchdog** ⓘ *a small check that reads the job's "heartbeat" file and raises an alert if a run went silent, a planned Monday never started, or nothing has run for 8 days*. |

The schedule never runs out. The workbook's Mondays come first; after its last Monday, **every Monday counts as a launch** except dates listed under `skip:` in `config/launch_schedule.yml` (a holiday week, say), which an import of a new workbook keeps. launchd starts the job every Monday at 17:30; the job itself decides whether today is a planned launch. On a server the same two slots are `Mon 12:00` and `Fri 03:30` UTC (`deploy/etl-loop.sh`).

> **One run at a time**
>
> Every program that writes pipeline data (the scheduled run, a hand load, serving and cube rebuilds, publish, integrity repairs) takes the same **run lock** ⓘ *a named lock held in the database (GET_LOCK 'cip_etl'); a second writer finds it taken and stops politely instead of colliding*. Two writers at once caused deadlocks on 20 Oct; this prevents it. The lock is held by a database connection, so a killed run frees it automatically, and the 54-hour watch keeps it alive with a tiny query every cycle.

## 3.2 A typical week



**So how fresh is the data?**

Because the watch polls every 30 minutes, a weekly wave is usually loaded **within half an hour of finishing**. With the median of about 13 hours in field, the data typically lands on **Tuesday morning IST**. **Trackers only:** Forsta sometimes leaves a full wave open (19 Oct: Midwest 80 of 85). So a weekly tracker counts as **fielded** ⓘ *fieldwork is over even though Forsta still says "live"* once any of these holds: **9 hours** since launch (about 03:00 IST), **400 qualified**, or every quota cell full. It is then loaded, marked "in field", and reloaded in place when Forsta closes it. **Annual and ad hoc studies never use this rule**: they load only once Forsta closes them, so they are never published half-fielded.

## 3.3 What one run does, in order

1. **Database health check.** `SELECT VERSION()`. If the database cannot be reached, it tries 4 times, 5 minutes apart, then stops with **exit code** ⓘ *a number a program returns when it ends: 0 means fine and anything else names the problem (see section 12)* 4.
2. **Key check.** `GET rh/users/self`. If Forsta rejects the key, it stops with exit 2.
3. **Schema check.** The newest tables (`cip_forsta_survey`, `cip_search`, `cip_tracker_line`) must exist. If not, exit 2. **Schema** ⓘ *the definition of the database tables and columns*
4. **Discover** every survey and mark what is due (section 6).
5. **Extract → Transform → Load → Verify → Derive** each due wave, oldest first (sections 6–10).
6. **Tell the team** about each new wave: email with the Excel files, plus a Teams message (section 10.2). If nothing was due, the run stops here (except on Friday).
7. **Integrity checks**, then **value-by-value reconciliation** of each newly loaded wave against Forsta (section 9).
8. **Topic dashboards** of the new waves, then the team's **metric alerts**.
9. **Publish** the portal's files (also after a failed reload, so no screen keeps stale numbers).
10. **Drift check.** Recompute every frozen published number. If one moved, exit 3. **Drift** ⓘ *a number already published to clients changing because the underlying data changed*

Everything is appended to `logs/forsta-etl.log`; every warning and error also goes to `logs/forsta-etl-errors.jsonl`, with secrets masked. A run that finds nothing new costs a handful of Forsta calls and two small queries.

# 4. The Forsta API: what we call

An **API** ⓘ *Application Programming Interface: web addresses that return data to programs instead of web pages to people* is how we read Forsta. Ours is Forsta's **REST** ⓘ *a style of API where each address is a "resource" and you use standard web verbs such as GET to read it* API, version 1.

## 4.1 Connection settings

| Setting | Value and meaning |
|---|---|
| Base URL | `https://se1.decipherinc.com/api/v1/`. Every call starts with this. **HTTPS** means the traffic is encrypted. |
| Method | **GET only.** **GET** ⓘ *the web verb that only reads; POST, PUT and DELETE would create, change or delete, and our client does not contain them* |
| Authentication | Header `x-apikey: <key>`. **Header** ⓘ *extra information sent with each request, alongside the address*. Also sent: `Accept: application/json` and `User-Agent: coresight-cip/1.0`. |
| Format | **JSON** ⓘ *JavaScript Object Notation, a text format of names and values in {braces} and [lists], readable by people and programs* |
| Timeout | 120 seconds per request (`FORSTA_TIMEOUT_SECONDS`). If Forsta does not answer in that time, the request is retried. |
| Retries | Up to **4 attempts**, waiting longer each time, from 2 s up to 60 s (**exponential back-off** ⓘ *wait, then wait about twice as long before each new try, so a busy server gets breathing room*). Retried on connection errors, timeouts, 429 and 5xx. |
| Usage | Forsta's `x-usage-today` header (calls used today) is written to the debug log. |
| Client code | `etl/forsta_client.py`, class `ForstaClient` |

## 4.2 The five endpoints, the only ones we use

An **endpoint** ⓘ *one specific API address that returns one kind of data*:

| Endpoint (all GET) | Returns | How often |
|---|---|---|
| `rh/users/self` | Who the key belongs to. Used only to prove the key works. | 1 per run |
| `rh/companies/all/surveys` | Every survey the key can see, with path, title, state, hibernated, tags, dates and counts (about 90 surveys). | 1 per run, plus 1 when something is due |
| `surveys/{path}/quota` | How full each **quota cell** ⓘ *a target group, e.g. "Midwest: 85 people"* is. Read only for surveys that are *live*, to decide whether a tracker wave has finished fielding; also shown on the portal's Launch schedule page. | 1 per live survey per run |
| `surveys/{path}/datamap?format=json` | The questionnaire definition (≈0.5 MB). | 1 per wave loaded |

| Endpoint (all GET) | Returns | How often |
|---|---|---|
| `surveys/{path}/data?format=json` | Every respondent's answers (≈2 MB per 500 respondents). We send **no** `cond` filter, so all statuses come back: terminated, over quota, qualified and partial. | 2 per wave loaded (the load, then the value check); 1 per wave in a reconciliation |

**Calls per run**

A run with nothing due makes **2 calls**, plus one quota call per live survey. A run that loads one weekly wave makes about **6 calls**: key, list, list again for the load, datamap, data, and data once more for the value-by-value check. The watch stops polling as soon as the week's wave is loaded and closed.

### 4.3 What Forsta's answers mean

Every reply carries an **HTTP status code** ⓘ *a three-digit number with every web reply: 2xx success, 4xx a problem with our request, 5xx a problem on the server* :

| Code | Meaning | What the pipeline does |
|---|---|---|
| **200** | OK | Carries on. |
| **401** | Key missing, wrong, revoked, or the user is disabled | Stops the run, exit 2. A person must renew the key. |
| **403** | Key valid but no rights on this survey | This wave fails and is recorded; the others continue. |
| **428** | Survey hibernated | Never loaded. Left alone; reconciliation skips it with a warning. |
| **429** | Too many requests (**rate limit** ⓘ *the server's cap on calls per period* ) | Waits and retries, up to 4 attempts. |
| **5xx** | Forsta server error | Retries up to 4 times. If it still fails, the wave is marked *failed* and retried at the next run. |

## 5. What data we receive

Forsta sends three kinds of JSON. Below is one real example of each; comments in yellow are ours.

### 5.1 The survey list: one item per survey

```
{ "path": "selfserve/58f/260908",        // unique survey address
  "title": "Shopping and Spending – inc Home Furnishings + Holiday Tracker + GenAI",
  "state": "closed",                       // testing | live | closed
  "hibernated": false,                     // true = data not readable
  "tags": ["Coresight Research", "Weekly consumer tracker"],
  "createdOn":    "2026-09-22T15:57:00Z",    // Z = UTC time
  "dateLaunched": "2026-10-05T12:22:25Z",
  "closedDate":   "2026-10-06T14:19:55Z",
  "qualified": 401,  "total": 823,         // finished & counted / everyone who started
  "sampleSources": ["114"] }               // the respondent panel used
```

We use this to decide **what to download**. It is stored in the register table `cip_forsta_survey`. A **register** ⓘ *one row per Forsta survey recording what Forsta says about it and what the pipeline has done with it*

## 5.2 The datamap: the questionnaire

```
{ "questions": [
  { "qlabel": "q1",                          // question code
    "qtitle": "Which, if any, of these have you done in the past two weeks?",
    "type": "multiple",                      // single | multiple | number | float | text
    "grouping": "rows",
    "variables": [                          // one per answer column in the data
      { "label": "q1r1", "rowTitle": "Met up with friends", "type": "multiple" },
      { "label": "q1r2", "rowTitle": "Eaten at a restaurant", ... } ] },
  { "qlabel": "D1", "qtitle": "What is your gender?", "type": "single",
    "values": [ {"value": 1, "title": "Male"}, {"value": 2, "title": "Female"}, ... ] },
  { "qlabel": "qtime", ..., "flags": ["t"] } ] }   // t = technical, v = virtual
```

A **variable** ⓘ *one column in Forsta's data; a single-choice question has one, a tick-all question has one per option (q1r1, q1r2…)* is the unit we store answers against. **Flags** ⓘ *markers on a question: "t" technical (timings, status, conditions) and "v" virtual (values Forsta computes, e.g. device or browser)* tell us which questions are not real survey questions.

## 5.3 The data: one object per respondent

```
[ { "record": "412", "uuid": "8k2f…",           // Forsta's ids for this attempt
    "status": "3",                              // 3 = qualified
    "date": "10/05/2026 14:07",                 // completion, MM/DD/YYYY HH:MM
    "qtime": "611",                             // seconds taken
    "q1r1": "1", "q1r2": "0",                 // ticked / shown but not ticked
    "D1": "2", "D2": "34", "D8": "TX",        // codes, numbers, text: ALL strings
    "q9oe": "Bought a new sofa",                // open-ended text
    "RID": "…", "userAgent": "…", "url": "…", "session": "…", "dcua": "…"  // personal: dropped or hashed
  }, ... ]
```

> **Important detail**
>
> **Every value arrives as text**, even numbers and codes ( `"3"` , `"34"` ). Part of the transform is converting each value to its proper type. An empty value means the question was not asked or not answered.

## 5.4 Size of a typical weekly wave

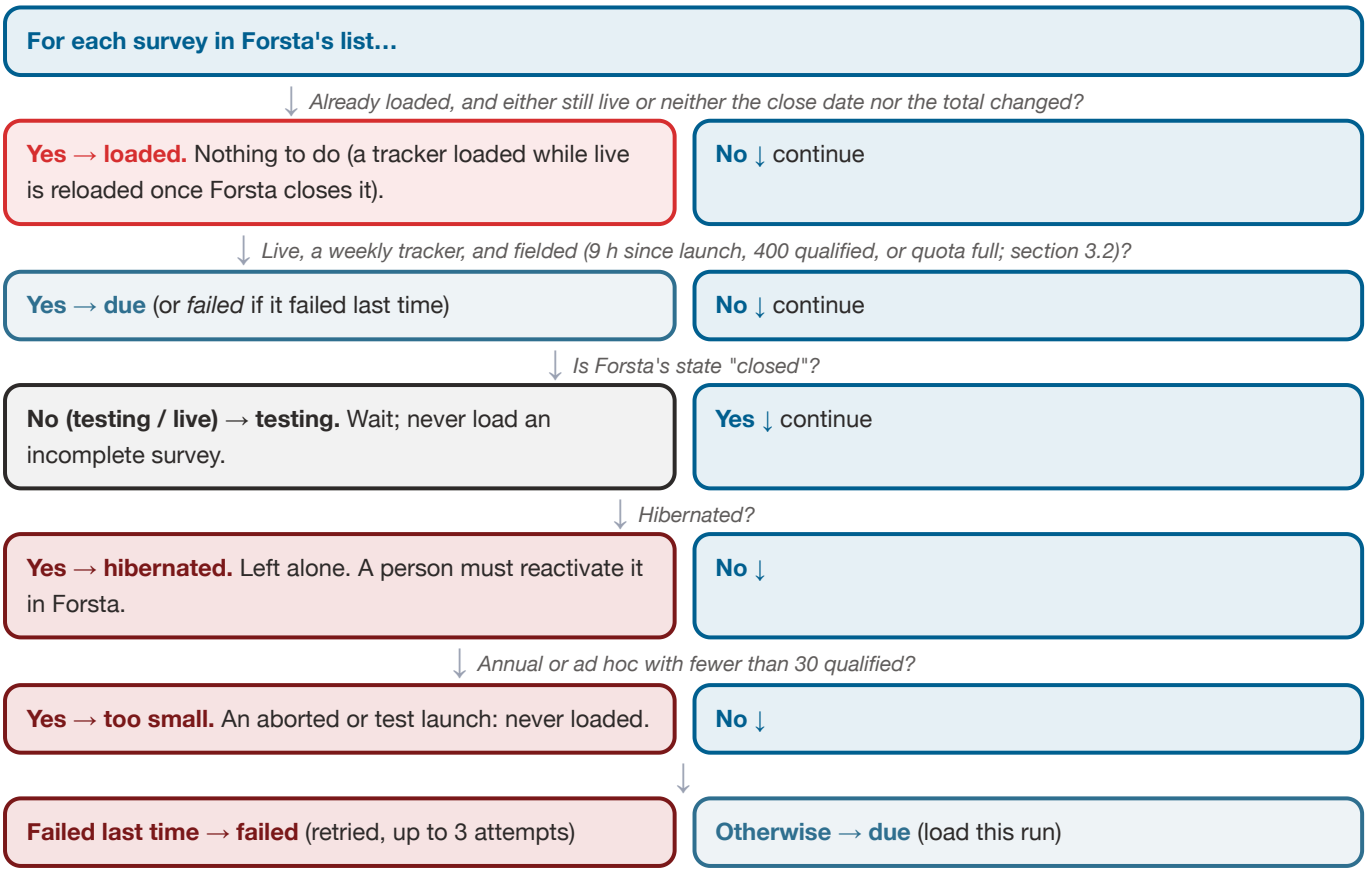
| Measure | Typical value |
|---|---|
| Respondents in the payload (all statuses) | 403 – 887 (≈400 of them qualified) |
| Questions per wave | ≈60 (1,416 measured across the loaded waves) |
| Answer rows stored per wave | ≈145,000 on average (4,476,153 across 31 Forsta waves) |
| Download size | datamap ≈0.5 MB + data ≈2 MB |
| Load time | Usually 1.5 – 3 minutes for the load step (219 s measured on 23 Oct, see section 14), mostly ≈250 ms database round trips from India to Azure |

# 6. Extract: finding and fetching surveys

**What happens:** the pipeline first lists everything, decides which surveys are ready, then downloads each ready one in full.

## 6.1 Step 1: discovery, on every run

**Discovery** ⓘ *the step that asks Forsta for the full survey list and updates the register; it downloads no answers* calls `rh/companies/all/surveys` once and **upserts** ⓘ *"update or insert": change the row if it exists, add it if not, so running twice never creates duplicates* one row per survey into `cip_forsta_survey`, keyed by the path. It then gives each survey a **load state** ⓘ *the pipeline's own label for a survey: testing, due, loaded, failed or hibernated* :




**Reopened surveys:** if a loaded survey's close date or total changes (it was reopened and more people answered), it becomes *due* again and is reloaded in place.

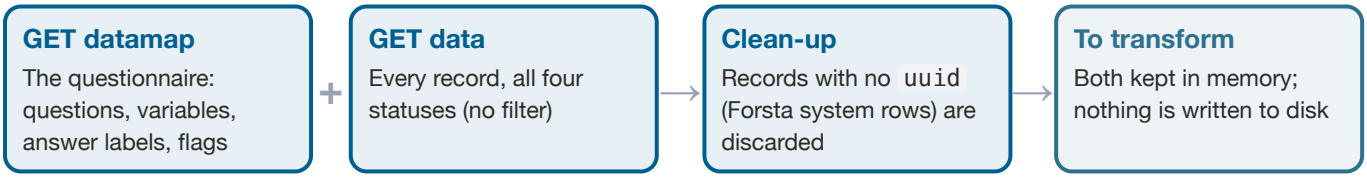
Every change of state is also written to a history table, `cip_forsta_event`, and each live survey's quota progress to the register, for the portal's Launch schedule page.

Discovery also records `study_type`: **tracker** if the survey is tagged "Weekly consumer tracker" or titled "Shopping and spending…", **annual** if tagged "annual tracker", otherwise **adhoc**. In the same step it rebuilds the survey calendar (section 10).

## 6.2 Step 2: choose what to load

All *due* surveys, and *failed* ones with fewer than **3 attempts**, are taken **oldest close date first** and loaded **one at a time**. If one fails, its error is saved in `last_error` and the run moves on to the next. One bad survey never blocks the others. A survey that has failed 3 times in a row waits for a person to run it by hand (section 12), instead of a full download every 30 minutes.

## 6.3 Step 3: download the wave



Why fetch every status, not only qualified? Partial and terminated records show where people dropped out and let us check our counts against Forsta's `total`. Reports count **qualified respondents only**.

### 6.4 Step 4: work out which week it is

| Item | Rule |
|---|---|
| Wave date / label | The **Monday** of the earliest completion date among **qualified** records (all records if none qualified). The wave launched Mon 5 Oct 2026 becomes `2026-10-05`. |
| Reload | A survey this pipeline has loaded before (load log `forsta:{path}`) keeps its wave label and internal `survey_id`, even if its first date moves. |
| Family | `CSI-US` for the weekly tracker. For other studies, the family listed for that survey in `config/survey_map.yml` (so "Holiday Shopping and Spending 2025" and "Holiday Shopping 2026" are both `HOLIDAY_SHOPPING`); if it is not listed, the title without the year in capitals. **Family** ⓘ *the group of waves that are compared with each other over time* |

## 7. Transform: turning JSON into rows

**What happens:** the datamap becomes **question definitions** and each record becomes **one respondent row plus one answer row per answered variable**. Personal data is removed at this point, before anything reaches the database.




### 7.1 Recognising the question type

Forsta's `type` and `grouping` decide how a question is stored. Each question becomes our `qtype`:

| Forsta shape | Seen | Stored as | Meaning and example |
|---|---|---|---|
| `single`, 1 variable | 693 | **single** | **Single choice** ⓘ *pick exactly one answer*. D1 gender: one field `D1`, value 1–4. |
| `multiple` | 491 | **multi** | **Tick-all / checkbox** ⓘ *pick any number of items; each item is its own yes/no column*. q1 activities: fields `q1r1`, `q1r2` … each 0 or 1. |
| `single`, rows, many variables | 73 | **grid_single** | **Grid** ⓘ *several statements (rows), each answered on the same scale (columns)*. One field per row; row label "row — column". |
| `single` + a text variable | 53 | **single** + **text** | A choice question with an "Other (please specify)" box. The box becomes its own text question (see 7.2). |
| `single`, rows, 1 variable | 43 | single or grid | Grid if the variable's name differs from the question code. |
| `number` / `float` | 32 | **numeric** | A typed number, such as an amount spent. |
| `text` / `textarea` | 23 | **text** | **Verbatim** ⓘ *the respondent's own words, stored exactly as typed*. |

## 7.2 The rules applied to every question

| # | Rule | What it does |
|---|---|---|
| 1 | Clean wording | HTML codes such as `&` become normal characters and extra spaces are removed. A short label of 80 characters or fewer is made for charts. |
| 2 | Answer labels | Taken from the question's `values`. In about a third of datamaps these are empty, so they are taken from the first variable that has them. Only whole-number codes are kept. |
| 3 | Grid detection | Spots a single-choice question that has several rows. |
| 4 | Bipolar grid | **Bipolar grid** ⓘ *each row is a pair of opposite statements and the respondent picks one side, e.g. "I'll shop early" vs "I'll shop late"*. Forsta leaves the answer titles blank, so we store 1 = left statement and 2 = right statement, and save both statements per row. The old Excel route lost about 3,300 of these answers. |
| 5 | Other (specify) | The text box inside a choice question becomes its own text question, e.g. `q1r2oe`, worded "<question> — <row> (please specify)". |
| 6 | Technical / virtual | Flag `t` (timings, status, conditions) or `v` (device, browser, panel list), or a title ending "-NORMALIZED". These are **stored but never reported**. |
| 7 | Personal fields | `RID`, `userAgent`, `url`, `session`, `dcua` and `ipAddress` never become questions, even if the datamap lists them. |
| 8 | Topic | Each question is placed in an editorial topic (demographics, holiday, tariffs…) from its code and wording, using `config/survey_map.yml`. |
| 9 | Answer flags | "Don't know / Not sure / Prefer not to say" are marked **non-response**. "None of these" is marked **exclusive**. Item codes ending `oe` are marked **other**. |

## 7.3 Each record → one respondent row

| We store | From Forsta | Rule |
|---|---|---|
| `record_no`, `forsta_uuid` | `record`, `uuid` | Forsta's own ids, used as the key so reloads update rather than duplicate. |
| `status_code`, `status_label`, `is_qualified` | `status` | 1 Terminated · 2 Overquota · 3 Qualified · 4 Partial. Qualified = status 3. |
| `completed_at` | `date` | `MM/DD/YYYY HH:MM` text → a real date-time. |
| `interview_secs` | `qtime` | Whole seconds. |
| `device`, `os`, `browser`, `dropout_qcode` | `vmobiledevice`, `vos`, `vbrowser`, `vdropout` | Short category labels, at most 50 characters (not the raw browser string). |
| `panel_source` | `vlist` / `list` | Which panel supplied the person. |
| `respondent_key` | `RID` | **SHA-256 hash** ⓘ *a one-way scramble: the same input always gives the same 64-character code, but the code cannot be turned back into the panel id*. The real panel id is never stored. |

### 7.4 Each answered variable → one answer row

Answers are stored **long, not wide** ⓘ *one row per person per variable, instead of one column per question; the table never needs new columns when weekly modules change* in `cip_answer`:

| Variable kind | Stored as |
|---|---|
| Code (single, grid row, checkbox item) | `value_code` as a whole number (`"3"` and `"3.0"` both become 3), plus `value_label`, e.g. "Female". |
| Code that is not a whole number | The raw text goes in `value_label`; no code is invented. |
| Number | `value_number`, only if it is a real finite number below $10^{14}$, **rounded to the 4 decimals the column holds** (a timing such as `12.642187` is stored as 12.6422). Interview seconds are rounded the same way. |
| Text | `value_text`, exactly as typed. |
| Empty | **No row.** "Not answered" means no row exists. |
| Checkbox "0" | **Stored.** It means "shown but not ticked", and it gives every tick-all question its base of people who answered. |

**Example: one record becomes rows**

| Forsta value | field | value_code | value_number | value_text / label |
|---|---|---|---|---|
| `"D1": "2"` | D1 | 2 | | Female |
| `"q1r1": "1"` | q1r1 | 1 | | Met up with friends |
| `"q1r2": "0"` | q1r2 | 0 | | Eaten at a restaurant (not ticked) |
| `"D2": "34"` | D2 | 34 | | 34 |
| `"q9oe": "Bought a new sofa"` | q9oe | | | Bought a new sofa |
| `"q7": ""` | *no row: not asked or not answered* | | | |
| `"userAgent": "Mozilla…"` | *dropped: personal* | | | |

# 8. Load: writing to the database

**What happens:** the transformed wave is written into `cip_stg` in a fixed order. The step that replaces respondents and answers is a single **transaction** ⓘ *a group of database writes that either all succeed together or are all undone together, so nobody ever sees half a wave*.

1 **Survey row** in `cip_survey`, set to `load_status = loading` (not visible in reports).

↓

2 **Run log** row in `cip_load_log`, with `source_ref = forsta:{path}`. This is the audit trail of every run. **Load log** ⓘ *a diary of every load: what, when, rows read and rows written, success or error*

↓

3 **Definitions**: topics, questions, options, items and fields (`cip_topic`, `cip_question`, `cip_option`, `cip_item`, `cip_field`). One batched statement per table.

↓

4 **Transaction A, all or nothing:**
a) upsert every respondent into `cip_respondent`, keyed by survey + record;
b) delete respondents Forsta no longer sends, with their answers, profile and cohort rows;
c) delete the wave's old answers, then upload the new ones in **one LOAD DATA** ⓘ *MySQL's bulk loader: the rows travel as one compact tab-separated file instead of thousands of INSERT statements* statement. If the database reports **any warning at all**, even a mild "note", the whole transaction is undone and the wave fails: a value is never silently changed.

↓

5 **Verify** (section 9). If anything is off, the wave is marked *failed*.

↓

6 **Tidy the answer list**: an answer row the questionnaire no longer lists **and nobody chose** is removed (8.2).

↓

7–13 **Derive**: profiles, question bases, concepts, cohorts and cube, search, line-by-line, serving rows (section 10).

↓

14–16 **Finish**: log row closed with row counts. `cip_survey.load_status = verified`, register set to *loaded*, calendar rebuilt, and next week's and next year's waves refreshed so they compare with this one.

## 8.1 Why it is safe to run again and again

### Idempotent

**Idempotent** ⓘ *running the same load twice gives exactly the same result as running it once*. Every write is an upsert or a replace keyed on Forsta's own ids (path, uuid, record, variable label), so nothing is ever duplicated.

### No half-loaded waves

If transaction A fails, the previous answers are untouched. A wave that was already verified stays visible with its old, correct data.

### Why one bulk upload?

Each database call from India to Azure takes about 250 ms. Writing 145,000 answers one by one would take 10 hours. One LOAD DATA upload sends them all at once; the whole write step on 23 Oct (693 respondents and 128,665 answers) took 20.6 s. The upload file is temporary and deleted straight away. If the server refuses bulk uploads, the loader falls back to batches of 5,000 rows.

## 8.2 What changed in the load since 14 Oct

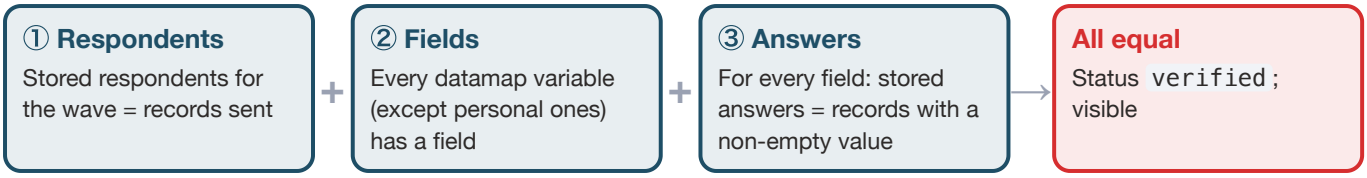
| Change | Why it matters |
|---|---|
| **Decimals rounded to 4 places** | MySQL stores 4 decimals. A longer value (a Forsta timing like `qtime` on 20 Oct) used to make the database round it with a warning, and the strict warning gate would then fail the whole wave on every attempt. Rounding first means a long decimal no longer fails a wave. |
| **Strict warning gate** | Any LOAD DATA warning undoes the write. Better a loud failure than a quietly altered number. |
| **Unlisted, unchosen answers removed** (`drop_unlisted_items`) | Definitions are only ever added or updated, never deleted. So when a questionnaire's answer list is corrected, the old rows used to linger as empty duplicates ("YouTube 0 %" next to "YouTube 47 %"). After the answers are written, a row the questionnaire no longer lists **and that nobody chose** (no code other than 0, no number, no text) is deleted with its "not chosen" zeros. A row with real answers is always kept. Both loaders (Forsta and legacy) do this. |
| **Legacy relabel** (surveys 97 and 201) | In two old waves the answer ids had shifted: SurveyMonkey survey 97 (US Social Commerce 2023) filed Facebook's 818 picks under "LinkedIn", and Qualtrics survey 201 (tracker, 15 Aug 2022) had 88 shifted grocery-retailer answers covering 8,029 picks. The legacy loader now gives each answer the one text its respondents actually saw. A scan of all 287 legacy waves found nothing else. After the reload, survey 97's Q7 shows Facebook 818 (63.3 %) and YouTube 612 (47.3 %). |
| **Failed reload drops its serving rows** | If a reload fails after its write, that wave's serving rows are removed, so no screen or published file keeps numbers for answers that have changed (section 12). |

## 8.3 Where it lands: the main tables

| Table | One row per | Per weekly wave |
|---|---|---|
| `cip_forsta_survey` | Forsta survey (the register) | 1 |
| `cip_survey` | Wave | 1 |
| `cip_question` / `cip_option` / `cip_item` / `cip_field` | Question / answer option / grid row or checkbox item / Forsta variable | ≈60 questions, hundreds of fields |
| `cip_respondent` | Person who started | 403 – 887 |
| `cip_answer` | Person × variable answered | ≈145,000 |
| `cip_load_log` | Load run | 1 per load |
| `srv_wave` / `srv_result` | Served wave / finished cell (answer × cut, with %, prior, last year and flags) | 1 / ≈12,000 (12,425 for wave 415) |

# 9. Verify: proving nothing was lost

**What happens:** before a wave can appear anywhere, the pipeline re-reads what it stored and compares it with the payload it downloaded. **Payload** ⓘ *the exact data Forsta sent in this run*




**Any difference**

The error "`{path} does not match its payload`" is saved in `cip_load_log.error_text` and `cip_forsta_survey.last_error`. The wave is marked **failed**, stays hidden and is retried at the next run.

## 9.1 Two gates since 23 Oct: the count, then every value

The count gate above proves nothing was *lost*. It cannot see a value that was stored *wrong*. So once a wave is in, the weekly run downloads its data from Forsta a second time and compares **every stored answer with Forsta, value by value**: codes exactly, numbers to 4 decimals, texts exactly. It costs one more download of the wave from Forsta.

| Result | What the run does |
|---|---|
| All answers equal | Nothing; the wave stays published. |
| Some answers differ | Counted as an integrity problem: alert "integrity check: N problems" with the first differences. |
| **Nothing could be checked** (`checked == 0`, e.g. Forsta unavailable or hibernated) | Also a problem, never a silent pass. The same rule applies to `python -m etl.forsta_etl --reconcile`. |

## 9.2 Integrity checks

`etl/integrity.py` runs about 20 checks after every load and on every Friday. **Derived** problems are repaired on the spot (at most 20 waves per run, so a mass gap heals over a few runs); **orphan** rows are deleted; **base** problems are reported. Examples:

| Check | Kind |
|---|---|
| Verified wave without cube cells, search rows, or (tracker) line-by-line rows | derived, repaired |
| **Served wave whose serving rows differ from its cube cells** (cube cells must equal serving rows; checked for waves built in the last 7 days) | derived, rebuilt |
| Search, tracker-line or calendar row pointing at something deleted | orphan, deleted |
| Wave stuck *loading*; register says loaded but wave not verified; duplicate Forsta respondent; same panelist qualified twice | base, reported |
| Closed Forsta survey not loaded after 3 days; hibernated before it was loaded | reported |
| A planned tracker week (launch schedule) still not in the portal 4 days after its Monday | reported |

A problem it cannot repair ends the run with exit code 5 (section 12).

### 9.3 Visibility

| load_status | Meaning | In reports? |
|---|---|---|
| loading | Being written | No |
| verified | Proven equal to Forsta, with everything derived | **Yes** |
| failed | Did not match, or a later step failed | No |

### 9.4 Full reconciliation: every wave, by hand

**Reconciliation** ⓘ *re-downloading every loaded wave from Forsta and comparing every single stored answer with it* is run by hand for **every** loaded wave with `python scripts/reconcile.py` (or `python -m etl.forsta_etl --reconcile`). It compares codes exactly, numbers to 4 decimals and texts exactly. A hibernated survey cannot be checked and now counts as a failure, not a pass. The weekly run does the same for each new wave automatically (9.1).

> **Last result (8 Oct 2026)**
>
> **31 Forsta waves · 4,476,153 answers · 0 differences.** The 142 Qualtrics and 145 SurveyMonkey waves are checked against their `dwh_sm*` source the same way.

## 10. Derive: making it report-ready, publishing, telling the team

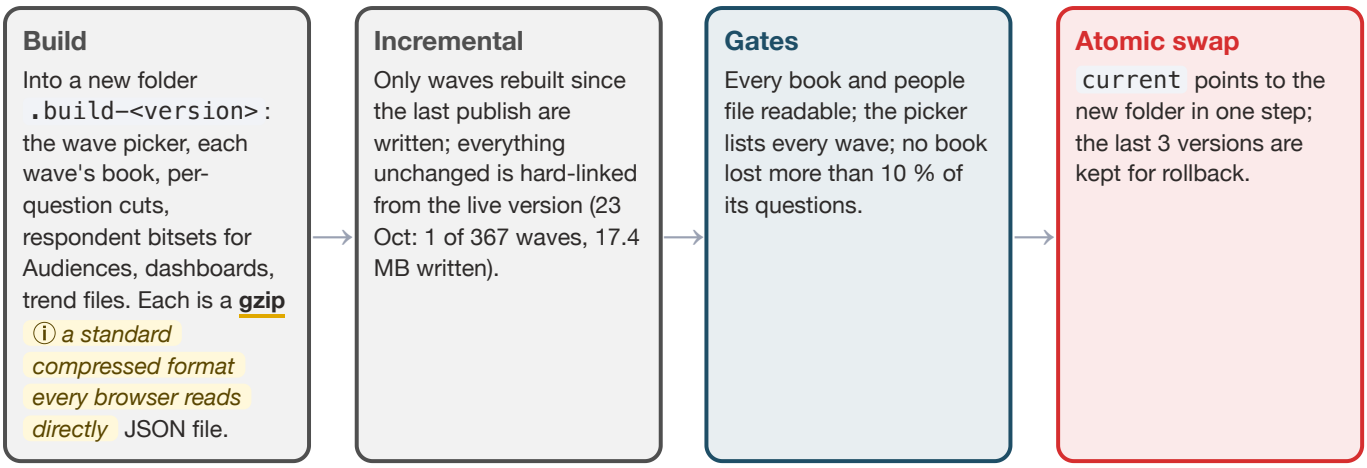
**What happens:** after verification, everything a report needs is calculated **once, at load time**, so the portal and exports only read finished numbers. Then the portal's files are published and the team is told.

| Output (table) | What it is and how it is built |
|---|---|
| **Profile**<br>`cip_profile` | One row per respondent with their demographics, read from D1 gender, D2 age, D3 relationship, D4 ethnicity, D5 income, D6 urban/suburban/rural, D7 politics, D8 state, CS1/CS2 outlook. Adds:<br>• **Age band**: 18-29 · 30-44 · 45-60 · Over 60 ("Under 18" is not banded)<br>• **Generation**: Gen Z 18-28 · Millennial 29-44 · Gen X 45-60 · Boomer 61+<br>• **Income midpoint** ($000): <$25k = 13 · $25–49.9k = 37.5 · $50–99.9k = 75 · $100–149.9k = 125 · $150–199.9k = 175 · $200k+ = 225<br>• **Census region**: state → Northeast / Midwest / South / West |
| **Question base**<br>`cip_question.base_n` | **Base** ⓘ *the number of people a percentage is calculated over: qualified respondents who were shown the question* |
| **Concepts**<br>`cip_concept*` | **Concept** ⓘ *"the same question" across waves and platforms, matched by wording because Forsta codes such as D2 or BT14 change meaning between waves*. Same wording and options → confirmed automatically. Changed options, or wording at least 85% similar → proposed for a person to confirm. Otherwise → a new concept. A one-off command (`python -m app.data.harmonise --confirm-same-wording`) also confirms waiting proposals whose wording and type **exactly** match their concept, when no other question in the wave uses it (1,322 units in the 23 Oct dry run); similar-wording proposals still wait for a person. Since 23 Oct confirmed concepts reach the serving rows, so a reworded but confirmed question keeps its prior-week and last-year comparison. |

| Output (table) | What it is and how it is built |
|---|---|
| **Cube**<br>`cip_agg_cell` | **Cube** ⓘ *a table of pre-counted totals: for every answer, how many people chose it, broken down by each standard cut*. Cuts: total, gender, age band, generation, income band, region, urbanicity, each for everyone and for each **cohort** ⓘ *a defined group of respondents, e.g. "Beauty shoppers"*. Qualified respondents only. |
| **Search**<br>`cip_search` | A full-text index of wave titles, question wording and concepts, so "GLP-1" finds every wave that asked about it. |
| **Line-by-line**<br>`cip_tracker_line` | The data team's "Weekly Line-By-Line Survey Data" sheet, rebuilt automatically: one row per qualified respondent in the team's column order. Checked against the team's file across 16 waves: identical apart from 3 errors in the team's file. |
| **Survey calendar**<br>`cip_wave_module` | Which sector, topics and seasonal trackers each wave carried, plus the holiday-season wave number. This enables "wave N vs last year's wave N" and predicts the next 12 lead sectors. |
| **Serving rows**<br>`srv_wave`, `srv_result`, `srv_answer` | The finished rows every screen reads, one per cube cell: labels, percentage, **prior week** and **same week last year**, and warning flags (small base under 50, wording changed, base shifted by more than 10 points). Rules since 23 Oct:<br>• **Cells are keyed by item and option id**, not by label, so two answers that share a label ("Other", or survey 97's two "YouTube" rows) are both served.<br>• **Prior and last year** are matched through the harmonised concept first, then by question code + normalised label + option code, and only when that match is unique in both waves.<br>• **Significance flags** `sig_prior` / `sig_ly`: a **two-proportion z-test** ⓘ *a standard test of whether two percentages differ by more than chance, here this week vs prior week (or last year)* on each Total row, then **Benjamini–Hochberg at 5 % FDR** ⓘ *a correction for testing hundreds of answers at once: it keeps the expected share of false "significant" calls at 5 %* over the wave's testable Total rows only. Testable means both percentages exist, **both bases are at least 30**, the base did not shift, and the answer is not a filler ("None of these", "Other –", "Prefer not…").<br>• **No blank segments**: respondents whose wave never asked a demographic are not shown as a segment equal to Total.<br>• **Excluded waves** (test runs and aborted launches listed under `exclude:` in `config/survey_map.yml`) and non-tracker waves under **30 qualified** get no serving rows: their raw data stays, but no screen lists them.<br>• **Annual studies** compare with their family's latest wave 300–430 days earlier (the family map), since they are not fielded in the same week each year.<br>• A tracker wave loaded while Forsta still had it open is marked **in field** until the reload after close. |
| **Dependants** | Next week's wave and next year's same wave are rebuilt, so they now compare with this one. |
| **Topic dashboards**<br>`cip_dash_*` | The **19** configured dashboards (`config/dashboards.yml`: holiday tracker, tariffs and inflation, GenAI, GLP-1, financial health, sector modules…) are rebuilt for the new waves that carry their module. A takeaway calls a change significant only when the serving row carries the `sig_*` flag. |

## 10.1 Publish: the portal's files

The data changes once a week, so the answers every visitor asks for are written to disk once, here, and served straight from disk by nginx: no database on the request path. `python -m etl.publish`:




A failed gate or a crash removes the half-built folder and leaves the previous version live. The newest waves are then pre-warmed in nginx's cache when that is configured.

### 10.2 Telling the team: email and Teams

As soon as a new wave is verified, before the checks and publishing that follow (section 3.3), `etl/notify.py` sends:

| Channel | What |
|---|---|
| **Email** | Subject "New survey ready: <title> (<wave>, <N> people)". A short summary: how many people completed it and in how many hours, that every answer was checked against Forsta, the biggest real changes against last week, and a "View in the portal" button. Attached: the team's Excel files (Cross Tabs, the Holiday Status and Inflation/Sentiment cross tabs when the wave carries them, and the Weekly Line-By-Line for the tracker): **4 files** for wave 415.<br>**To:** the 4 analysts (the `default` list in `config/notify.yml`); **Cc:** dataautomation@coresight.com, so the team has a record. |
| **Teams** | "✅ CIP pipeline succeeded: <title> (wave …) loaded and verified, N qualified. 4 files emailed." |
| **Failure alerts** | Email (same list until a `failure:` list is filled) and Teams. The subject carries the count ("integrity check: 3 problems"), so a new problem is not held back by the 6-hour repeat window. |

## 11. What we ingest and what we never store

### Ingested

| Data | From |
|---|---|
| Every survey's metadata: title, state, tags, dates, counts | survey list |
| Every question, answer option, grid row and variable | datamap |
| Every respondent of **every status** (1–4) | data |
| Every non-empty answer: codes, numbers, open text | data |

### Never stored

| Data | Why |
|---|---|
| `RID` (panel id) | Personal; stored only as a SHA-256 hash |
| `userAgent`, `dcua` | Browser fingerprint |
| `url`, `session` | Survey link and panel session |
| `ipAddress` | Personal |
| Raw JSON files | Kept in memory only; never written to disk (the answer upload uses a temporary file that is deleted at once) |

| Data | From |
|---|---|
| Completion time, interview length, device / OS / browser category, dropout question, panel source | data |
| Technical and virtual questions (stored, hidden) | datamap + data |

| Data | Why |
|---|---|
| Surveys still *testing*; annual / ad hoc studies still *live* | Incomplete; loaded once closed |
| Tracker waves still *live* before 9 h / 400 qualified / quota full | Still fielding; loaded once fielded, reloaded at close |
| *Too small* annual / ad hoc surveys (under 30 qualified) | Aborted or test launches |
| *Hibernated* surveys | Unreadable (428) until a person reactivates them |

### 11.1 Where things stand (23 Oct 2026)

| Source | Waves verified | Notes |
|---|---|---|
| Forsta (live) | 84 | Every closed Forsta survey is loaded, including the 2025 waves that had been hibernated (reactivated and loaded on 21 Oct). For all 84, Forsta's counts equal the stored respondents and the served counts. 8 surveys are still testing in Forsta. |
| Qualtrics (history) | 142 | 2022–2025 |
| SurveyMonkey (history) | 145 | 2018–2022 |
| **Total** | **371** | All verified; none loading or failed. 367 are published; excluded test waves and too-small studies are not. |

## 12. When something goes wrong

Every run ends with an exit code, written to the log, that says how it went:

| Code | Meaning | What to do |
|---|---|---|
| 0 | Everything due loaded and verified, or nothing was due | Nothing |
| 1 | A wave failed to load or verify | Read `cip_forsta_survey.last_error` (also in the alert). The next runs retry it automatically, up to 3 attempts. |
| 2 | Forsta rejected the key, the key is missing, or the database schema is older than the code | Renew the key in `.env`, or run `python scripts/init_db.py` |
| 3 | Loaded fine, but an already-published number changed | Review on the Publications page |
| 4 | Database unreachable after 4 tries over 15 minutes | Check the VPN or the Azure firewall (the office IP changed on 13 Oct). The next slot retries; during a Monday watch, the next 30-minute cycle. |

| Code | Meaning | What to do |
|---|---|---|
| 5 | Loaded fine, but an integrity check (or the value-by-value comparison) found something it could not repair | Read the "integrity check: N problems" alert; it lists each problem. |

## 12.1 Safety nets added since 14 Oct

| If… | …then |
|---|---|
| **One watch cycle crashes** (Forsta down past its retries, the database dropping mid-run) | The crash is logged and alerted ("a watch cycle crashed"), and **the watch carries on** 30 minutes later. Before, one crash ended the Monday watch and the wave waited for Friday. |
| The run stops without finishing (killed, Mac asleep or restarted) | The hourly watchdog sees the missed heartbeat and alerts; it also alerts if a planned Monday never started, or nothing has run for 8 days. |
| Something goes wrong | An alert by email and Teams, with the **problem count in the subject** ("integrity check: 3 problems", "published numbers moved in 2 publications"), so a new problem is told at once rather than after the 6-hour repeat window. |
| **A reload fails after its write** | That wave's serving rows are dropped and the portal is republished, so no screen or published file shows numbers for answers that have changed. A reload that fails *before* its write leaves the old, verified wave untouched. |
| **A publish fails** (a gate or a crash) | Its build folder is removed and the previous version stays live. It can never become a rollback target. |
| A published version turns out wrong | `python -m etl.publish --rollback` points `current` back to the previous version (the last 3 are kept). |
| A second writer starts while a run is going | It finds the run lock taken and stops (section 3.1). |

## 12.2 Common situations

| You see | Cause | Fix |
|---|---|---|
| "Loaded 0" for days | No survey has closed in Forsta | Expected |
| "Can't connect to MySQL … timed out" | VPN down, or the office IP is not allowed in Azure | Reconnect; the next slot retries |
| A wave "does not match its payload" | Forsta changed data mid-load, or a step broke | `python -m etl.forsta_etl --survey <path>` |
| "LOAD DATA into … warned …: nothing is kept" | The database would have changed a value | Read the warning; the old wave is untouched. Fix the cause, then reload the wave. |
| A wave stays *failed* after 3 attempts | The same error every time | Fix the cause, then `python -m etl.forsta_etl --survey <path>` |
| "Another CIP run holds the lock" | A run is already going | Wait until `logs/etl-heartbeat.json` says `finished` |
| Survey *hibernated* and its history is needed | Forsta hibernated it before it was loaded | A Forsta owner reactivates it; the next run loads it |
| Survey reopened and closed again | New respondents | Automatic: becomes due and is reloaded in place |

## 12.3 Commands to run by hand

Each command that writes takes the run lock itself; never start one while a scheduled run is going.

```
python scripts/weekly_forsta.py                          # the scheduled run, now
python -m etl.forsta_etl --discover                      # refresh the register only (no answers
downloaded)
python -m etl.forsta_etl --due                           # discover + load everything due
python -m etl.forsta_etl --survey selfserve/58f/260908   # load or reload one wave
python -m etl.forsta_etl --reconcile                     # compare every loaded wave with Forsta
python -m etl.serving --wave 415                         # rebuild one wave's serving rows
python -m etl.integrity --check                          # integrity checks, report only
python -m etl.publish                                    # build, check and swap the portal's files
python -m etl.publish --rollback                         # back to the previous published version
python scripts/weekly_forsta.py --watchdog               # the hourly heartbeat check, now
python scripts/test_connection.py                        # database + Forsta check (never prints the
key)
```

# 13. Worked example: one wave, start to finish

The wave `selfserve/58f/261001` (internal id 415), "Shopping and Spending – inc Social Commerce + Holiday Tracker + GLP1s". The timings are from a measured run on 23 Oct 2026, each stage run as its own process (section 14).

**Mon 19 Oct 2026, 17:30 IST.** A planned Monday in the launch schedule. Under today's schedule the job posts its Teams heads-up and starts watching: a run every 30 minutes.

↓

**12:27 UTC (17:57 IST).** Launched and *live*. Each poll reads its quota; not yet fielded, so it stays *testing*.

↓

**Mon 19 Oct, 18:35 UTC (Tue 20 Oct, 00:05 IST).** Closed at quota after about 6 hours: **400 qualified out of 693 started**.

↓

**Next 30-minute poll.** Health ✓, key ✓, schema ✓. Discovery: closed, not hibernated, a tracker → **due**. Under the tracker rule it would also have become due while still live, the moment it showed 400 qualified.

↓

**Extract (3.6 s).** GET datamap + GET data (693 records, all statuses). Rows with no uuid are discarded.

↓

**Transform. 58 reportable questions** typed: core q1–q6, CS1–CS3, D1–D8 plus extra D codes, SC1–SC10 social commerce, XM1–XM3 holiday tracker, GP1–GP10 GLP-1. Five "Other (specify)" boxes (e.g. `SC2r10oe`) become their own text questions. Personal fields dropped, RID hashed, every value converted from text to its type, decimals rounded to 4 places.

↓

**Load (definitions 10.8 s, write 20.6 s).** Survey row (*loading*) → log row → definitions → transaction A: 693 respondents, **128,665 answers** in one LOAD DATA upload, no warnings. Wave label = Monday **2026-10-19**.

↓

**Verify (2.7 s).** 693 = 693 respondents; every variable has a field; every field's answer count matches → ✓. Unlisted, unchosen answer rows tidied.

↓

**Derive.** Profiles and bases 10.0 s → harmonise 6.5 s → cube 113.9 s → search 4.4 s → **400** line-by-line rows 12.5 s → serving 18.2 s: **12,425 serving rows = 12,425 cube cells**; 12,183 of them carry a harmonised concept; **6** answers flagged significant against the prior week and **7** against last year. Then the dependants. Whole load step: **219 s**.

↓

**Dashboards (27 s) and publish (66 s).** The topic dashboards whose modules the wave carries; then an incremental publish: **1 of 367 waves rebuilt, 17.4 MB written**, gates passed, `current` swapped.

↓

**Team told (20 s).** Email "New survey ready … (2026-10-19, 400 people)" with **4 Excel files** to the 4 analysts, Cc dataautomation@coresight.com, plus the Teams message. Exit 0. Total ≈ **332 s (5.5 min)**; about 4–4.5 min with the usual cube time.

# 14. Timing and memory (measured 23 Oct 2026)

**How it was measured:** wave 415 (the 19 Oct tracker: 693 respondents, 400 qualified) was run tonight with **each stage as its own process**, and its memory sampled **every second**. **Peak memory** ⓘ *the most RAM the process held at any one moment; "average" is the mean over its run* is what a server must have free.

## 14.1 One weekly wave, stage by stage




Bar to scale: load · dashboards (27 s) · publish · email (20 s). Total ≈ 332 s.

| Stage | Time | Peak memory | Average | Notes |
|---|---|---|---|---|
| **Load** (extract → serving) | 219 s | 206 MB | 137 MB | Breakdown in 14.2 |
| **Dashboards** | 27 s | 55 MB | — | The topic dashboards the wave carries |
| **Publish** | 66 s | 173 MB | 98 MB | 1 of 367 waves rebuilt, 17.4 MB written |
| **Email** with 4 Excel files | 20 s | 228 MB | 181 MB | Plus the Teams message |
| **Total** | **≈ 332 s (5.5 min)** | About 4–4.5 min with the usual cube time (14.3) | | |

## 14.2 Inside the load step (seconds)

| fetch | definitions | write | verify | profiles | harmonise | cube | search | tracker line | serving |
|---|---|---|---|---|---|---|---|---|---|
| 3.6 | 10.8 | 20.6 | 2.7 | 10.0 | 6.5 | **113.9** | 4.4 | 12.5 | 18.2 |

## 14.3 A caveat about the cube

**Tonight's cube time is not the usual one**

The cube step took **27–42 s on 20–22 Oct**. Tonight it measured **about 110 s, three times**, with **unchanged code**, right after a full rebuild of all waves. Its parts add up to about **19 s** when timed in isolation, so the extra time is inside the shared Azure database, most likely its own cleanup after the mass rebuild. This cannot be confirmed: our database account lacks the PROCESS privilege needed to see what the server is doing. With the usual cube time the end-to-end run is **about 4–4.5 minutes**; on 22 Oct whole runs measured **243–264 s** end to end, peaking at **304–314 MB** in one process.

## 14.4 RAM and CPU: what IT needs to provide

| Question | Answer |
|---|---|
| Do the stages' memory figures add up? | **No.** Memory is held while a step runs and freed after it. The weekly job runs in **one process**, one step after another, so plan for the **peak, about 230–320 MB**, not a sum. Between polls nothing runs. |
| The portal API | 112–138 MB steady on its own; its answer cache is capped at 256 MB, so at most about 400 MB. |
| One VM for everything | ETL + API + nginx stays **under about 1 GB at peak. 2 GB is comfortable.** |

| Question | Answer |
|---|---|
| CPU | **1 vCPU is enough.** The ETL used less than 1 s of CPU per stage; about 98 % of its time is **waiting** on database round trips of about 0.25 s from India to Azure. |
| Biggest speed lever | Running the job on an Azure VM in the database's own region, so each round trip takes milliseconds instead of a quarter of a second. *This is an estimate, not yet measured.* |

## 14.5 Rare full rebuilds (not weekly)

Only needed when a rule changes for every wave (as on 22–23 Oct). The weekly run never does these.

| Rebuild | Time | Memory and size |
|---|---|---|
| Serving rows for all 371 waves (4 parallel jobs) | 23 min | peak 603 MB |
| All 19 dashboards | 15 min | — |
| Full publish (367 waves) | 16 min | 506.5 MB written; peak 314 MB |